

17.4.4 系统分析与设计

本节以一个多用途通用控制平台为例，对嵌入式系统的开发与设计过程进行描述。该平台是一个面向工业、企事业单位、生活社区和普通家庭用户，集成了移动通信、互联网技术、无线局域网和无线传感器网络等先进的网络技术，以及视频信息采集与处理技术的多用途控制平台。

对于家庭用户来说，可以通过移动通信网络随时随地了解家中状况，可以远程控制家里的电器设备，并且设备出现了故障或紧急情况，该通用控制平台可以主动地通过手机或电话网通知用户或火警等相关部门。特别是出现火灾等极端状况时，控制平台可以启动有关设备进行扑救，同时将相关数据和现场图像或视频资料发送出去，提供给救援人员参考，以便协助救援。对于企业用户，该系统可以设置在仓库、车间和实验室等地点，并且多个平台设备之间可以共享数据和相互协作。

1. 需求分析

嵌入式系统一般都是面向产品开发的，对嵌入式系统进行需求分析是一项复杂而费时的的工作。需求分析阶段最重要的成果之一就是系统规格说明书。对于不同的项目，该文档形式可以灵活变化，但基本包含物理尺寸、操作环境、存放环境、指示灯装置、无线电标准、无线电功率和频率、数据传输率、存储器、平均无故障时间、功耗、电源适配器、散热、系统复位、协议等内容。在进行需求分析时，可以采用 UML 进行需求描述。

2. 系统架构设计

嵌入式系统的架构由硬件架构和软件架构组成，随着嵌入式软件复杂度的日益提高，系统的响应速度和可靠性等重要指标不再仅由硬件架构所决定，软件架构的影响也逐步增大。描述系统如何实现规格说明中定义的功能是系统架构设计的主要目的。但是，在设计嵌入式系统的架构时，很难将软件和硬件完全分开。通常的处理方法是先考虑系统的软件架构，然后再考虑其硬件实现。系统架构的描述必须符合功能上和非功能上的需求，不仅要体现所要求的功能，还要满足成本、速度和功耗等非功能约束。

软件架构的选择和设计在很大程度上取决于嵌入式系统的架构，主要考虑以下几点因素：

(1) 系统的实时性要求。对于硬实时系统，必须进行严格的时序分析。

(2) 采用的设计模型。对于简单的中小型嵌入式系统，可以采用“先硬件后软件”的方法；对于复杂、大型嵌入式系统，必须采用软硬件协同设计的方法，还要充分考虑软件和硬件两方面的互相制约和影响。

(3) 是否需要 EOS。对于复杂的或者日后需要继续开发和移植的系统，最好采用 EOS 来增强系统的可扩展性。

在所有满足系统功能和性能要求的方案中，应该优先选用最简单的架构。

3. 硬件子系统设计

嵌入式系统的开发环境由 4 个部分组成，分别是目标硬件平台、EOS、编程语言和开发工具。其中，处理器和操作系统的选择应当考虑更多的因素，避免错误的决策影响项目的进度。

嵌入式系统设计的主要挑战是如何使互相竞争的设计指标同时达到最佳化。设计人员必须

对各种处理器技术和 IC (Integrated Circuit, 集成电路) 技术的优缺点加以取舍。一般而言, 处理器技术与 IC 技术无关。也就是说, 任何处理器技术都可以使用任何 IC 技术来实现, 但是最终器件的性能、一次性工程 (Non-Recurring Engineering, NRE) 成本、功耗和大小等指标会有很大的差异。通用的可编程技术提供了较大的灵活性, 降低了 NRE 成本, 建立产品样机与上市的时间较快; 定制的技术能够提供较低的功耗、较好的性能、更小的体积和大批量生产时的低成本。根据这些原则, 设计人员便可以对采用的处理器技术和处理器做出合理选择。

一般地, 全定制商用“通用处理器 + 软件”是大多数情况下都适用的一个选择。根据用户的需求和项目的需要, 选择合适的通用嵌入式处理器, 选择时需要考虑处理器的速度、技术指标、开发人员对处理器的熟悉程度、处理器的 I/O 功能是否满足系统的需求、处理器的调试、处理器制造商的支持可信度和封装形式等指标。

在硬件子系统的设计中, 首先, 将硬件划分为部件 (或模块), 并绘制部件连接框图; 其次, 对每个部件进行细化, 将系统分成更多可管理的小块, 可以被单独实现。通常, 系统的某些功能既可用软件实现也可用硬件实现, 没有一个统一的方法指导设计人员决定功能的软硬件分配, 但是可以根据系统约束清单, 在性能和成本之间进行权衡。

嵌入式系统中接口电路的设计需要考虑的因素主要有电平匹配问题、驱动能力和干扰问题。设计时需要注意 I/O 端口、硬件寄存器、内存映射、硬件中断和存储器空间分配等方面。总之, 硬件设计人员应该给软件设计人员更多、更详细的信息, 便于进行软件设计和开发。

4. 软件子系统设计

根据需求分析阶段的规格说明文档, 确定系统计算模型, 然后, 对软件部分进行分析与设计。嵌入式系统的软件设计需要根据应用的实时性要求、应用场合、可用资源情况等诸多约束, 进行合理的任务划分, 这直接影响软件设计的质量。其步骤如下:

(1) 首先, 明确系统的实时性指标。究竟是软实时还是硬实时, 最坏情况下的实时时限是多少等, 以及确定任务大体的数目。任务划分的力度和数目的合理性直接决定着调度开销和任务之间的通信开销, 以及系统响应时间的重要决定因素, 这需要充分考虑极端情况下的各种因素。实时系统的软件应该尽可能简化, 过于复杂的软件设计会增加系统的复杂性, 从而降低可靠性。

(2) 其次, 进行可调度性分析。对已经划分好的任务根据实现的功能特点进行分类, 识别出具有硬实时性要求的所有关键任务, 并给所有的任务赋予适当任务优先级, 根据每个任务执行的时间估计值在考虑调度开销余量的条件下进行粗略的可调度性分析, 从而确保所有关键任务都满足调度时限。根据分析结果再对所有的任务进行适当的分裂与重组。对于周期相同的所有任务的功能进行适当的组合形成一个单独的任务, 以降低事件分发机制的开销; 对于若干固定顺序执行的任务要合成一个单一任务, 避免同步开销; 将若干有相同的事件源触发的任务也合并成一个任务, 以减小事件分发机制的开销; 将计算密集型或以数据处理为主而不进行 I/O 操作的功能独立出来, 由一个优先级的任务来实现; 对任务之间的同步与互斥机制进行分析, 尽量避免大粒度的互斥锁。

(3) 最后, 在具体的实现上, 根据系统特点确定任务的优先级。任务优先级分配的一般原则如下:

- 与中断的关联性。凡是与中断服务程序相关联的任务应该安排尽量高的优先级，以保证及时处理异步事件，提高系统的实时响应能力。否则，CPU可能被其他优先级高的任务长期占用，使得第二次中断发生时，上一次中断还没有处理，从而产生信号丢失，甚至是外部设备失败。
- 紧迫性。越是紧迫的任务对响应时间的要求就越严格，对于所有的紧迫任务，根据它们响应时间的先后顺序，安排优先级。
- 任务的频繁性。对于周期性的任务，执行越频繁，则周期越短，允许耽搁的时间也就越短，故相应的优先级也应该越高，以保证处理的及时性。并且，任务的执行时间越短，优先级也应越高。
- 传递性。信息处理流程的前驱任务的优先级应该高于后继任务的优先级。例如，数据的采集任务优先级应高于数据处理任务的优先级。

5. EOS 的选择

在选择 EOS 时，也需要做如下多方面的考虑：

(1) EOS 的功能。根据项目需要的 EOS 功能来选择 EOS 产品，要考虑系统支持 EOS 的全部功能还是部分功能，是否支持文件系统和人机界面，是实时系统还是分时系统，以及系统是否可裁减等因素。

(2) 配套开发工具。有些 RTOS 只支持该系统供应商的开发工具。也就是说，还必须向 EOS 供应商获取编译器和调试器等；有些 EOS 使用广泛，且有第三方工具可用，因此选择的余地比较大。

(3) EOS 的可移植性。EOS 到硬件的移植是一个重要的问题，是整个系统能否按期完工的关键因素。因此，要选择那些可移植性程度高的 EOS，从而避免 EOS 难以向硬件移植而带来的种种困难，加速系统的开发进度。

(4) EOS 的内存需求。均衡考虑是否需要额外 RAM 或 EEPROM 来迎合 EOS 对内存的较大要求。有些 EOS 对内存的要求是目标相关的，如 Tornado/VxWorks 等，开发人员能按照应用需求分配所需的资源，而不是为 EOS 分配资源。

(5) EOS 附加软件包。EOS 是否包含所需的软件部件，如网络协议栈、文件系统和各种常用外设的驱动等。

(6) EOS 的实时性如何。有些 EOS 只能提供软实时性能，对于需要达到硬实时性能要求的系统就不适用；有些 EOS 既可满足软实时要求，又能满足硬实时要求，如 MS Windows CE 2.0 等。

(7) EOS 的灵活性。EOS 是否具有可剪裁性，即能否根据实际需要进行系统功能的剪裁。有些 EOS 具有较强的可剪裁性，如嵌入式 Linux 和 ECos 等。

6. 编程语言的选择

对于嵌入式系统编程语言的选择，需要考虑通用性、可移植性、执行效率和可维护性等方面。

(1) 通用性。随着微处理器技术的不断发展，其功能越来越专业，种类越来越多，但不同

种类的微处理器都有自己专用的汇编语言。这就给系统开发人员造成了一个巨大的障碍，使系统编程更加困难，软件复用无法实现；而高级语言一般和具体机器的硬件结构联系较少，比较流行的高级语言对多数微处理器都有良好的支持，通用性较好。

(2) 可移植性。由于汇编语言和具体的微处理器密切相关，为某个微处理器设计的程序不能直接移植到另一个不同种类的微处理器上使用，因此可移植性差；高级语言对所有微处理器都是通用的，因此程序可以在不同的微处理器上运行，可移植性较好。这是实现软件复用的基础。

(3) 执行效率。一般来说，越是高级的语言，其编译器和开销就越大，应用程序也就越大、越慢。但单纯依靠低级语言（如汇编语言）来进行应用程序的开发，带来的问题是编程复杂、开发周期长。因此，存在一个开发时间和运行性能之间的权衡。

(4) 可维护性。通常，低级语言程序的可维护性不高。高级语言程序往往是模块化设计，各个模块之间的接口是固定的。因此，当系统出现问题时，可以很快地将问题定位到某个模块内，并尽快得到解决。另外，模块化设计也便于系统功能的扩展和升级。

在嵌入式系统的分析与设计过程中，需要完成一系列的文档，如技术文件目录、技术任务书、技术方案报告、产品规格、技术条件、设计说明书、试验报告和总结报告等，这些文档对完成产品设计和维护相当重要，需要按照通用计算机系统开发的文档管理标准进行管理。

17.4.5 低功耗设计

嵌入式系统通常都是一些移动设备，不能像通用计算机系统那样，长期插接供电电源。因此，在进行嵌入式系统设计时，必须考虑其功耗问题。事实上，低功耗设计是嵌入式系统设计中的难点，是一个系统化的综合问题，必须从软件和硬件两方面全面考虑，才能真正有效地降低功耗。

1. 基于硬件的低功耗设计

对于常用的典型 CMOS 集成电路，其功耗由静态功耗、静态漏电流功耗、内部短路功耗和动态功耗组成。通常，在 CMOS 器件的整体功耗中，动态功耗大约占 70% ~ 90%，静态功耗与静态漏电流功耗一般不大于 2%，内部短路功耗在 10% ~ 30% 之间。从硬件方面降低器件工作时的功耗，主要从降低工作电压方面考虑。对于嵌入式系统设计人员来说，基于硬件的低功耗设计应该从如下几方面考虑：

(1) 板级电路低功耗设计。板级电路的低功耗设计主要围绕处理器的低功耗特性和外围芯片的工作特点。

(2) 选择低功耗处理器。处理器是嵌入式系统不可缺少的核心部件，选择处理器时除了参考其功能、性能、接口和指令集等指标外，还应该考虑功耗特性，在满足正常工作的前提下，尽量选择低电压工作的处理器。

(3) 总线的低功耗设计。对于嵌入式系统来说，总线的宽度越宽，功耗就越大，在可能情况下，应该优先选用低功耗的总线器件，如低电压差分信号器件（Low Voltage Differential Signal, LVDS）。

(4) 接口驱动电路的设计。设计接口驱动电路时，要选用静态电流低的外围芯片，还要考